Listing 6.7 `agentic_btree_coding_challenge.py`

```
root = py_trees.composites.Sequence("RootSequence", memory=True)


thread = api.create_thread()     #1
challenge = textwrap.dedent("""
 #2
""")
judge_test_cases = textwrap.dedent("""
 #3
""")

hacker = create_assistant_action_on_thread(
    thread=thread,      #4
    action_name="Hacker",
    assistant_name="Python Coding Assistant",
    assistant_instructions=textwrap.dedent(f"""
    Challenge goal:
    {challenge}      #5
    Solve the challenge and output the
final solution to a file called solution.py
    """),
)
root.add_child(hacker)

judge = create_assistant_action_on_thread(
    thread=thread,      #6
    action_name="Judge solution",
    assistant_name="Coding Challenge Judge",
    assistant_instructions=textwrap.dedent(
        f"""
    Challenge goal:
    {challenge}      #7
    Load the solution from the file solution.py.
    Then confirm is a solution to the challenge
and test it with the following test cases:
    {judge_test_cases}      #8
    Run the code for the solution and confirm it passes all the test cases.
    If the solution passes all tests save the solution to a file called
judged_solution.py
    """,
    ),
)
root.add_child(judge)

# verifier operates on a different thread, essentially in closed room
verifier = create_assistant_condition(     #9
    condition_name="Verify solution",
    assistant_name="Python Coding Assistant",
    assistant_instructions=textwrap.dedent(
        f"""
    Challenge goal:
    {challenge}      #10
    Load the file called judged_solution.py and
verify that the solution is correct by running the code and confirm it passes
all the test cases:
    {judge_test_cases}      #11
    If the solution is correct, return only the single word SUCCESS, otherwise
return the single word FAILURE.
    """,
    ),
```